Reviewer Pack — Minimal Hodge Decomposition Rank and Circuit Monotone Width …

Entry a54005a18dca · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 03:32:34 UTC

Minimal Hodge Decomposition Rank and Circuit Monotone Width Inequality

Entry ID: `a54005a18dca` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 03:32:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Decomposition Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every d-regular graph G, the minimal Hodge decomposition rank (h(G)) of its associated real algebraic surface is lower-bounded by the monotone width w_m(G) of G's circuit representation, such that h(G) = Ω(w_m(G)).

Rationale (proposer's reasoning):

> The connection between Hodge theory and complexity may reveal new structures in circuits. The rank of a Hodge decomposition reflects the geometric complexity of an algebraic variety, which could potentially relate to the structural complexities of circuits.

Taxonomy category: `HODGE_DECOMPOSITION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `808999cbaba8ab53`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all 30 random seeds, the minimal Hodge decomposition rank h(G) of the real algebraic surface associated with graph G is at least twice the monotone width w_m(G) of G's circuit representation, i.e., h(G) >= 2 * w_m(G). The conjecture is falsified if any seed results in h(G) < 2 * w_m(G).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 5 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge decomposition rank" AND "circuit monotone width"` - `"real algebraic surface" AND d-regular graph` - `"minimal Hodge decomposition rank" <= "monotone width"`

Top relevant hits considered: - [http://arxiv.org/abs/math/0201057v2] Generalised Thurston-Bennequin invariants for real algebraic surface singularities - [http://arxiv.org/abs/2510.04025v1] On the non-existence of certain real algebraic surfaces - [http://arxiv.org/abs/1609.07446v2] On the geometric structure of certain real algebraic surfaces - [http://arxiv.org/abs/2007.03314v1] The exact solution to the Shallow water Equations Riemann problem at width jumps in rectangular channels - [http://arxiv.org/abs/2304.11307v1] Statistical Investigation of the Widths of Supra-arcade Downflows Observed During a Solar Flare

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            return None
        graph = {i: [] for i in range(n)}
        edges = set()
        while len(edges) < (n * d) // 2:
            u, v = random.sample(range(n), 2)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                graph[u].append(v)
                graph[v].append(u)
                edges.add((u, v))
        return graph

    def real_algebraic_surface(graph):
        n = len(graph)
        A = [[Fraction(0, 1)] * n for _ in range(n)]
        for i in range(n):
            for j in graph[i]:
                if i < j:
```

```
                A[i][j] = Fraction(1, 2)
                A[j][i] = Fraction(1, 2)
    return A

def hodge_rank(A):
    n = len(A)
    rank = 0
    for _ in range(n):
        max_row = -1
        max_val = -math.inf
        for i in range(n):
            if all(A[i][j] == Fraction(0, 1) for j in range(i)):
                if A[i][i] > max_val:
                    max_row = i
                    max_val = A[i][i]
        if max_row != -1:
            rank += 1
            for j in range(n):
                A[max_row][j] = Fraction(0, 1)
                A[j][max_row] = Fraction(0, 1)
    return rank

def monotone_width(circuit):
    n = len(circuit)
    dp = [0] * (n + 1)
    for i in range(n):
        dp[i + 1] = max(dp[j] for j in range(i) if circuit[j][1] < circuit[i][1]) + 1
    return dp[-1]

def circuit_representation(graph):
    n = len(graph)
    circuit = []
    visited = [False] * n
    stack = []
    for i in range(n):
        if not visited[i]:
            stack.append(i)
            while stack:
                u = stack.pop()
                if not visited[u]:
                    visited[u] = True
                    for v in graph[u]:
                        if not visited[v]:
                            circuit.append((u, v))
                            stack.append(v)
    return circuit

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_max = 0
    instances_tested = 0
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""
```

```
        for n in [5, 10, 15, 20, 30, 40]:
            if time.time() + (n - 5) * 8 > 240:
                print('RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30')
                return
            graph = generate_d_regular_graph(n, 3)
            if graph is None:
                continue
            A = real_algebraic_surface(graph)
            h_G = hodge_rank(A)
            circuit = circuit_representation(graph)
            w_m = monotone_width(circuit)
            if h_G < 2 * w_m:
                conjecture_holds = False
                counterexample = f"h(G)={h_G}, w_m(G)={w_m}"
            total_metric_value += h_G
            instances_tested += 1
            n_max = max(n_max, n)

        return {
            "metric_name": "h(G)",
            "metric_value": total_metric_value / instances_tested,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

    return run_trial(seed)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results) and any(r["instances_tested"] >= 30 for r
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None
RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30
TRIAL: None

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_528251a4.py", line 135, in <module>
    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_528251a4.py", line 135, in <genexpr>
    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
                            ~^^^^^^^^^^^^^^^^
TypeError: 'NoneType' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data for all trials, which means that the conjecture could not be tested as intended. | next: Investigate and fix the crash in the test code to allow for proper testing of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14318
2	preregistration	ollama_remote	glm4:latest	0	0	11062
3	novelty	ollama_remote	glm4:latest	0	0	8296
4	novelty	ollama_remote	glm4:latest	0	0	9422
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12579
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14627
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12117
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15544
9	judge	ollama_remote	glm4:latest	0	0	12667

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 110632 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a54005a18dca.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a54005a18dca.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a54005a18dca.tar.gz` (if generated)